

Современные архитектуры нейронных сетей

Прокопов Е.М.

Содержание

1	Введение в глубокое обучение	4
1.1	Что такое глубокое обучение?	4
1.2	Базовые архитектуры нейронных сетей	4
1.2.1	Полносвязные нейронные сети	4
1.2.2	Функции активации	6
1.2.3	Свёрточные нейронные сети	9
1.2.4	Рекуррентные нейронные сети	10
1.3	Обучение нейронных сетей	11
1.3.1	Функции потерь	11
1.3.2	Обратное распространение ошибки	15
1.3.3	Градиентный спуск	17
1.3.4	Регуляризация	21
1.3.5	Нормализация	22
I	Трансформеры	24
2	Токенизация	24
2.1	Посимвольная токенизация	24
2.2	Токенизация по словам	24
2.3	Byte-pair encoding (BPE)	25
2.4	Embedding-слой	26
3	Трансформер	27
3.1	Преобразование одной последовательности в другую	27
3.2	Внимание (Attention)	28
3.2.1	Мультипликативное внимание (Multiplicative attention)	28
3.2.2	Самовнимание (Scaled-dot Self-Attention)	28
3.2.3	Многоглавое внимание (Multihead Attention)	32
3.2.4	Маскированное многоглавое внимание (Masked Multihead Attention)	34
3.2.5	Перекрытое внимание (Cross Attention)	34
3.3	FFN-слой	35
3.4	Нормализация	35
3.5	Слой трансформера	36
3.5.1	Остаточные связи (Residual connections)	36
3.6	Позиционное кодирование	37
3.6.1	Абсолютное позиционное кодирование	38
3.6.2	Синусоидальное кодирование	38
3.7	Трансформер кодировщик и трансформер декодировщик	39
3.8	Обучение трансформера	41
3.8.1	Perplexity	42
3.9	Инференс трансформера	43
3.9.1	Температура трансформера	44

3.10	Масштабируемость трансформеров	44
4	Vision Transformer	44
4.1	Проблемы свёрточных нейронных сетей	44
4.2	Токенизация изображений	45
4.3	Обучаемое абсолютное позиционное кодирование	46
4.4	CLS токен	46

1 Введение в глубокое обучение

1.1 Что такое глубокое обучение?

Несколько десятилетий назад, на заре зарождения компьютерных технологий решение задач компьютерами было строго алгоритмическим: после запуска программы следом за выполнением первой, заранее прописанной инструкции, следовало выполнение следующей, также заранее прописанной инструкции. И так до окончания работы программы.

Логика вызова этих инструкций определялась с помощью логических конструкций на основе простейших логических операций `if`, `else`, операторов `and`, `or` и так далее.

Такой подход способен автоматизировать множество задач. Например, таким образом возможно реализовать логику работы манипулятора на конвейерном производстве. Однако, таким образом очень сложно определять в данных какие либо паттерны, закономерности.

Оцифрованных данных в свою очередь, по мере развития технологий, становилось все больше и больше. Если есть аналитики, способные по определенным данным прогнозировать, например, стоимость недвижимости, почему нельзя научить это делать компьютер, используя статистические методы (как, например, линейная регрессия)?

Такие алгоритмы, которые вместо использования жестких, заранее прописанных правил, способны обучаться на имеющихся данных и извлекать из них закономерности (зачастую даже не видные человеку), являются алгоритмами машинного обучения.

Машинное обучение делится на несколько разделов, из них можно выделить два основных: обучение с учителем и обучение без учителя.

В ходе обучения с учителем модель учится на парах данных “вход” (обозначим его x) и “правильный ответ” (обозначим его y). Такие данные называются размеченными, поскольку для определения правильного ответа при составлении набора данных требуется участие эксперта-разметчика (однако, есть и исключения, о которых мы поговорим позднее).

При обучении без учителя модель работает с не размеченными данными и самостоятельно пытается обнаружить в них закономерности. Глубокое обучение, наоборот, по большей части, является подмножеством обучения с учителем. Оно основано на глубоких, то есть многослойных, нейронных сетях. Глубокие нейронные сети позволяют определять сложные высокоуровневые признаки через распознавание на каждом слое более простых низкоуровневых признаков и их комбинаций.

1.2 Базовые архитектуры нейронных сетей

1.2.1 Полносвязные нейронные сети

Наиболее распространенным слоем среди нейронных сетей является полносвязный линейный слой. В этом слое каждый нейрон связан с каждым

нейроном предыдущего слоя. Связи между нейронами, веса, являются обучаемыми параметрами нейронной сети. Каждый нейрон вычисляет взвешенную сумму входов:

$$y_j = \sum_i x_i w_{ij}$$

Запишем выход полносвязного слоя в виде вектора y :

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_d \end{pmatrix}^T = \begin{pmatrix} \sum_{j=1} x_j w_{1j} \\ \sum_{j=1} x_j w_{2j} \\ \dots \\ \sum_{j=1} x_j w_{dj} \end{pmatrix}^T = xW$$

Рассматривая выражение $y = xW$ можно заметить, что оно описывает гиперплоскость, проходящую через начало координат. Поэтому, при $x = 0$ всегда $y \equiv 0$ при любых параметрах. Соответственно, при входе $x = 0$ результат глубокой сети, состоящей из полносвязных слоёв, всегда будет нулевым. Добавив в выражение вектор $b \neq 0$ обучаемых параметров

$$y = xW + b,$$

мы получим гиперплоскость, способную проходить где угодно в пространстве признаков.

Попробуем построить сеть из двух слоёв f_1 и f_2 , описываемых параметрами $\{W_1, b_1\}$ и $\{W_2, b_2\}$ соответственно:

$$y_1 = xW_1 + b_1, \quad y_2 = y_1W_2 + b_2$$

Подставим y_1 в выражение для y_2 :

$$y_2 = (xW_1 + b_1)W_2 + b_2 = xW_1W_2 + b_1W_2 + b_2 = x\tilde{W} + \tilde{b}$$

Мы получили из двух слоёв один. Получается, это значит, что глубокие нейронные сети невозможны? "Схлопывание" слоёв происходит из-за линейности функции $y = xW + b$. Для линейной функции выполняется выражение

$$f(ax + by) \equiv af(x) + bf(y),$$

а композиция линейных функций — линейная функция. Поэтому, чтобы сделать глубокие сети возможными, добавим в композицию слоёв нелинейную функцию ϕ :

$$y_1 = \phi(f_1(x)), \quad y_2 = \phi(f_2(y_1))$$

После подстановки, выражение не удастся упростить:

$$y_2 = \phi(\phi(xW_1 + b_1)W_2 + b_2)$$

Нелинейную функцию ϕ называют функцией активации. Нелинейность функции активации также позволяет нейронной сети проще аппроксимировать сложные нелинейные преобразования. От функции активации также должна существовать производная, это свойство потребуется нам в дальнейшем.

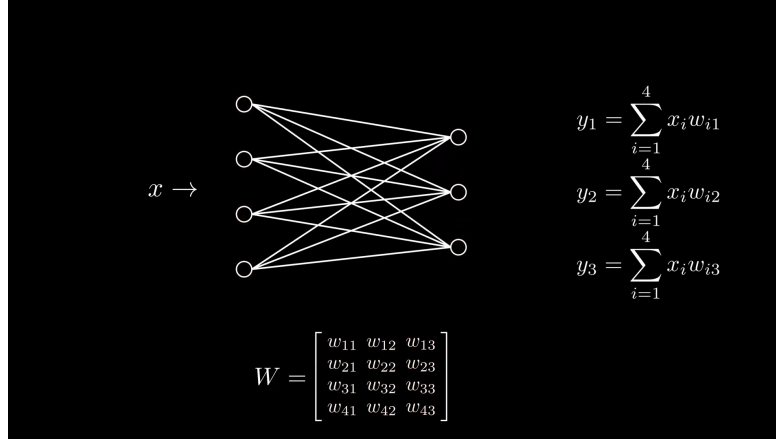


Рис. 1: Схема полносвязного слоя.

Полносвязная нейронная сеть (fully connected network) также называется многослойным перцептроном (multilayer perceptron, MLP) или сетью прямого распространения (feedforward network, FFN).

1.2.2 Функции активации

Исторически, одними из первых широко использовавшихся функций были сигмоида:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

и гиперболический тангенс:

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

Однако обе функции имеют большой недостаток: при больших по модулю входных значениях они "насыщаются", то есть их значение почти не изменяется при изменении аргумента. Из-за этого их производная будет близка к нулю. О том, почему это плохо, мы поговорим далее.

Сигмоида отражает входное значение в диапазон от 0 до 1, благодаря чему его удобно интерпретировать как вероятность.

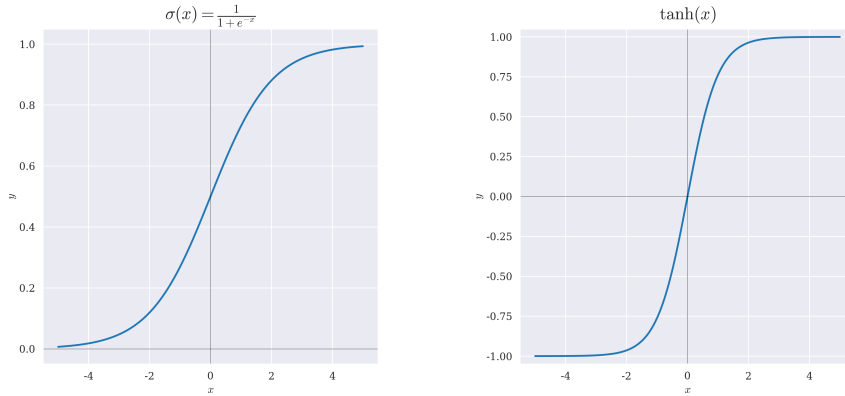


Рис. 2: Графики функций активации сигмоиды (слева) и гиперболического тангенса (справа).

На смену сигмоиде и гиперболическому тангенсу в качестве функции активации пришла функция ReLU (Rectified Linera Unit):

$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0, \\ 0, & x < 0 \end{cases}$$

ReLU нелинейна на $x \in \mathbb{R}$, даже несмотря на линейность лучах $x \geq 0$ и $x < 0$. Ее главное преимущество заключается в простоте вычислений, а также в простоте производной. Но в производной кроется и её главный недостаток: при $x < 0$ производная $\text{ReLU}'(x) \equiv 0$.

Эта проблема решается в модификации ReLU — Leaky ReLU:

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \quad \alpha \neq 0 \end{cases}$$

Обычно, коэффициент α берут небольшим, например $\alpha = 0.1$. Так мы сохраняем нелинейность функции и решаем проблему нулевого градиента.

В PReLU идея развивается дальше и коэффициент α становится обучаемым параметром.

Такие функции как ELU устроены сложнее:

$$\text{ELU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha * (\exp(x) - 1), & x < 0 \end{cases}$$

Их идея заключается в том, чтобы сделать отрицательную часть не линейной, а плавной, чтобы получить более гладкую функцию и, за счёт этого, улучшить динамику обучения (правда, пожертвовав простотой вычислений).

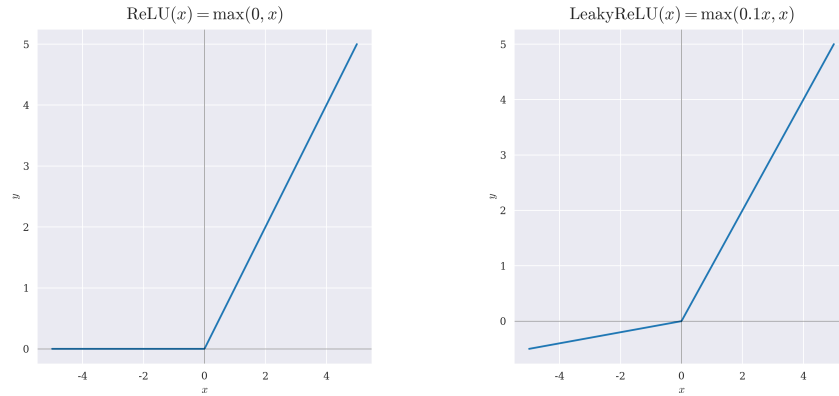


Рис. 3: Графики функций активации ReLU (слева) и LeakyReLU (справа).

Широкое распространение также получила функция SiLU, поскольку является непрерывно гладкой на $\mathbb{R}$:

$$\text{SiLU}(x) = x\sigma(x)$$

И ее обобщение — функция активации SiLU_β :

$$\text{SiLU}_\beta(x) = x\sigma(\beta x)$$

Устремление коэффициента β к бесконечности позволяет превратить SiLU_β в ReLU.

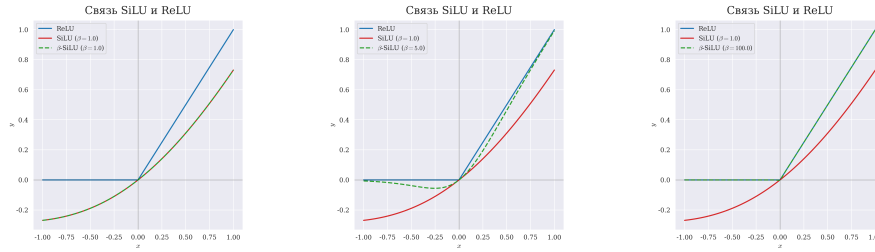


Рис. 4: График функции активации SiLU_β при разных значениях β .

Отдельно стоит рассмотреть функцию Softmax. Формально, она также является функцией активации:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{\dim(x)} \exp(x_j)}$$

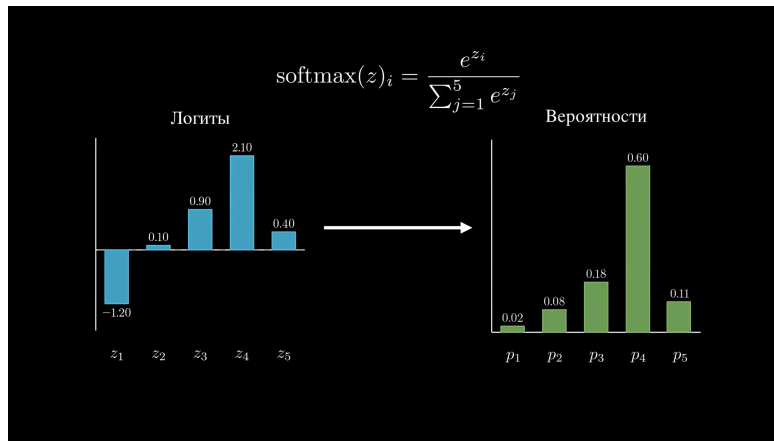


Рис. 5: Преобразование распределения логитов в распределение вероятностей с помощью Softmax.

Её главная особенность заключается в том, что сумма всех компонент вектора, пропущенного через Softmax, равна 1, что позволяет трактовать этот вектор как многомерное распределение вероятностей. Softmax обычно используют при решении задачи многоклассовой классификации.

1.2.3 Свёрточные нейронные сети

Полносвязные нейронные сети не учитывают пространственных связей в данных. Например в таких, как изображения или текст. Во время обработки, изображение выпрямляется в вектор, после чего каждая компонента обрабатывается как отдельный признак. При этом, если перемешать пиксели изображения, а также соответствующим образом перемешать веса нейронной сети, результат обработки не изменится.

Для обработки таких данных, в которых важна информация о пространственном взаиморасположении признаков, используются свёрточные нейронные сети (convolutional neural network, CNN). Такие сети состоят из свёрточных слоёв. Эти слои применяют к данным фильтры - однослойные полносвязные сети, преобразующие участок данных в число.

В одномерном, простейшем, случае при обработке последовательности данных фильтром размера 3, на первом шаге обрабатываются первые 3 элемента, после чего фильтр смещается на некоторый шаг свёртки (stride) дальше и обрабатывает ещё 3 элемента последовательности. Когда последовательность завершается, из результатов обработки формируется карта признаков — новая последовательность, которую также можно обработать свёрточным слоём.

При обработке двумерного одноканального изображения обрабатывается участок размером 3×3 . В этом случае сеть-фильтр имеет 9 параметров (10 в случае наличия смещения). Когда фильтр доходит до края изобра-

жения, он опускается вниз на шаг свёртки и смещается к левому краю. По завершению обработки двумерных данных карта активации получается двумерной.

При обработке многоканального изображения, каждый канал сначала обрабатывается по отдельности, с использованием своего фильтра, а затем полученные карты активаций складываются. Для получения ещё одной карты признаков исходные каналы обрабатываются независимо с использованием новых фильтров.

Важной особенностью свёрточных слоёв является то, что они применяют одинаковый набор параметров к каждому участку изображения. Таким образом, если на изображении присутствует полезный признак (например, наличие вертикальных или горизонтальных линий), то неважно, в какой части он находится, он все равно будет обработан одинаково. Это свойство называется инвариантностью сдвига.

Для свёрточных сетей, также как и для полносвязных, важно использование функции активации.

Отдельно стоит упомянуть выбор размера фильтра. Оказывается, что при использовании соответствующего размера паддинга (заполнения нулями краёв изображения), последовательная обработка изображения двумя свёрточными слоями с фильтрами 3×3 может быть сведена к обработке изображения свёрточным слоем с фильтром 5×5 и наоборот. Аналогично с тремя слоями с фильтрами 3×3 и слоем с фильтром 7×7 . Однако, в первом случае возможно использование дополнительно двух функций активации и всего 20 параметров, а во втором — только одной функции активации при 26 параметрах. Таким образом очевидно, что выгоднее использовать два слоя с фильтрами 3×3 , чем один слой с фильтрами 5×5 , поскольку большее число функций активации способствует большей выразительности нейронной сети. При этом два таких слоя выгоднее по количеству параметров сети.

1.2.4 Рекуррентные нейронные сети

Рекуррентные нейронные сети (RNN) предназначены для работы с последовательными данными, где присутствует зависимость между текущим входом и предыдущими. Такими данными могут быть, например, текст или временные ряды.

$$\begin{cases} h_t = \phi(x_t W + h_{t-1} U + b_1), \\ y_t = \varphi(h_t V + b_2) \end{cases}$$

Главная особенность RNN - наличие обратных связей: сеть способна передать информацию из предыдущих элементов последовательности. Это можно представить в следующем виде:

W , U , V - матрицы параметров, ϕ , φ - активации, b_1 , b_2 - смещения.

h_t - в некотором смысле “память” о предыдущих элементах последовательности. Однако “память” RNN очень короткая и не способна запоминать

долгосрочные зависимости. С этой задачей лучше справляется другая рекуррентная архитектура - LSTM.

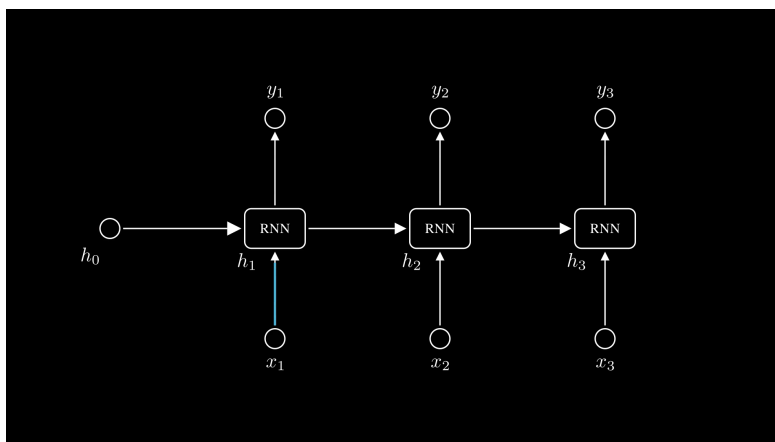


Рис. 6: Схема RNN слоя.

1.3 Обучение нейронных сетей

Нейронные сети обучаются, изменяя свои параметры таким образом, чтобы минимизировать функцию потерь L . Задача обучения становится оптимизационной задачей, и для ее решения будем использовать инструменты из математической оптимизации, а конкретно градиентные оптимизаторы.

Процесс обучения можно разделить на следующие этапы: прямое распространение (forward pass), вычисление **функции потерь** (loss-функции) и обратное распространение (backpropagation) для обновления весов. Данный алгоритм обучения называется алгоритмом **обратного распространения ошибки**.

1.3.1 Функции потерь

Функция потерь — это оценка стоимости ошибки модели. Пусть y — истинные значения, а $\hat{y}$ — предсказания модели. Простейший способ оценить ошибку — это взять модуль разности по всем объектам и просуммировать их:

$$\text{Absolute Error}(y, \hat{y}) = \sum_{i=1}^N |y_i - \hat{y}_i|$$

Поскольку перед нами стоит задача оптимизации, мы можем применять монотонные возрастающие преобразования и точки минимумов и максимумов не изменятся. Поэтому, разделим абсолютную ошибку на количество

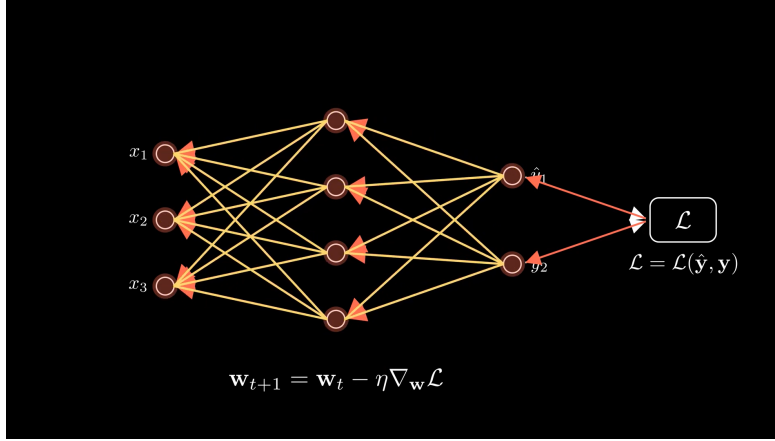


Рис. 7: Обратное распространение ошибки.

объектов, по которым считали ошибку, и получим среднюю абсолютную ошибку (Mean Absolute Error, MAE):

$$\text{MAE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Также используется средне квадратичная ошибка в качестве функции потерь (Mean Squared Error, MSE). Эта функция сильнее штрафует большие ошибки и слабее — малые.

$$\text{MSE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Какую же функцию потерь выбрать? Пусть данные устроены следующим образом:

$$y = f(x) + \varepsilon,$$

где ε - случайная величина, шум.

Предположим, что шум центрирован и распределен нормально:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Если мы зафиксируем значение x , то значение y тоже будет распределено нормально:

$$y|x \sim \mathcal{N}(f(x), \sigma^2)$$

Запишем плотность такого распределения:

$$p(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{y - f_w(x)}{\sigma} \right)^2 \right]$$

Теперь нам нужно понять, какая функция будет лучше соответствовать модели шума ε . Для этого воспользуемся методом максимального правдоподобия.

Необходимо максимизировать величину, называемую правдоподобием:

$$l = \prod_{i=1}^N p(y_i|x_i)$$

Правдоподобие показывает, насколько хорошо выбранные параметры объясняют наблюдаемые данные. В случае, если ответы y - дискретная величина, мы можем говорить, что правдоподобие равно произведению вероятностей выбора правильного ответа моделью.

Произведение оптимизировать неудобно, поэтому вместо него рассматривают логарифм правдоподобия, чтобы превратить произведение в сумму. Также, обычно рассматривается не задача максимизации $\log(l)$, а задача минимизации $-\log(l)$:

$$\text{NLL} = -\sum_{i=1}^N \log p(y_i|x_i) = -\log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{y - f_w(x)}{\sigma} \right)^2 \right] \right)$$

Упростим:

$$\text{NLL} = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - f(x))^2}{2\sigma^2}$$

Здесь $\frac{1}{2} \log(2\pi\sigma^2)$ - это константа, поэтому от нее можно избавиться.

Остаток домножим на $\frac{2\sigma^2}{N}$ (потому что можем это сделать) и получим эквивалентную задачу оптимизации

$$w^* = \operatorname{argmin}_w (\text{NLL}) = \operatorname{argmin}_w \left(\frac{1}{N} \sum_{i=1}^N (y - \hat{y})^2 \right)$$

Таким образом, если шум в данных распределен нормально, то наиболее оптимальная функция потерь — это MSE. Если же шум в данных имеет распределение Лапласа, то наиболее оптимальной функцией потерь будет MAE.

В задаче регрессии шум чаще всего распределен нормально. Шум, распределенный Лапласово, встречается, также может встретиться в задаче регрессии, но чаще его можно обнаружить, например, в задаче генерации изображений.

А вот для задачи классификации не подойдут ни MAE, ни MSE, поскольку шум в такой задаче имеет категориальное распределение. Чтобы

определить оптимальную функцию потерь, воспользуемся теорией информации.

Пусть имеется C классов. Тогда для объекта x модель f выдает вектор вероятностей

$$q(x) = (q_1(x), q_2(x), \dots, q_C(x))$$

Если некоторое событие X произошло с вероятностью p , то количество информации, которое несёт это событие (мера самоинформации), равно

$$I[X] = -\log p$$

отсюда видно, что маловероятное событие несет больше информации, чем вероятное. Математическое ожидание информации в одном наблюдении называется энтропией (Шенноновской энтропией):

$$H(p) = \mathbb{E}[I[X]] = -\sum_{i=1}^C p_i \log p_i$$

Теперь предположим, что истинное распределение классов — p , а модель предсказывает распределение q . Тогда среднее количество информации, которое потребуется для описания истинных ответов с использованием распределения q , задаётся кросс-энтропией:

$$H(p, q) = -\sum_{i=1}^C p_i \log q_i$$

В задаче обучения с учителем p — это вектор, у которого $p_c = 1$ для правильного класса, а для остальных $p_i = 0$. Тогда можем упростить:

$$H(p, q) = -\log q_c,$$

где q_c — вероятность, которую модель присвоила правильному классу.

Если мы решим применить метод максимального правдоподобия для решения задачи классификации, то для одного объекта получим:

$$\text{NLL} = -\log q_c = H(p, q)$$

Значит мы можем записать функцию потерь кросс-энтропии:

$$L_{CE} = \frac{1}{N} \sum_{n=1}^N H(p^{(n)}, q^{(n)}) = -\frac{1}{N} \sum_{n=1}^N \log q_{c^{(n)}}^{(n)}$$

Но что эта функция значит? Какая интуиция? Вернемся к полной формуле кросс-энтропии и немного перепишем её:

$$H(p, q) = -\sum_i p_i \log q_i = \underbrace{-\sum_i p_i \log p_i}_{H(p)} + \underbrace{\sum_i p_i \log \frac{p_i}{q_i}}_{D_{KL}(p||q)},$$

Таким образом:

$$H(p, q) = H(p) + D_{KL}\langle p||q \rangle,$$

где:

- $D_{KL}\langle p||q \rangle$, — расхождение Кульбака-Лейблера, "расстояние" от распределения p к распределению q (на самом деле это не расстояние, поскольку расхождение Кульбака-Лейблера не обладает свойством симметричности).
- $H(p)$ — Шеннонова энтропия данных, мера "шума". Это — теоретический минимум для корректной функции потерь.

Таким образом, минимизация кросс-энтропии означает минимизацию расхождения Кульбака-Лейблера, а значит приближает моделируемое распределение q к истинному распределению p .

1.3.2 Обратное распространение ошибки

Пусть f - функция, которая представляет нашу нейронную сеть, а $f_1, f_2, \dots, f_n$ — ее слои. Этап прямого распространения тогда записывается просто:

$$y = f(x) = f_1 \circ f_2 \circ \dots \circ f_n(x)$$

После прямого распространения и вычисления функции потерь, необходимо вычислить градиент $\nabla L(y, \hat{y})$. После чего используется алгоритм градиентного спуска для обновления параметров:

$$w_t = w_{t-1} - \lambda \nabla L_{w_{t-1}}(y, \hat{y})$$

Этап обратного распространения заключается в вычислении градиента ∇L_w . Градиент — это вектор частных производных:

$$\nabla L_w = \left(\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \dots \quad \frac{\partial L}{\partial w_N} \right)$$

Переменные $w_1, w_2, \dots$ — это параметры нейронной сети, а именно веса конкретных слоёв. Допустим, $w_1 = (W_1)_{ij}$ - это параметр слоя f_1 , а $w_2 = (W_2)_{ij}$ - параметр слоя f_2 .

Найдем $\frac{\partial L}{\partial w_1}$. Для этого продифференцируем сложную функцию с помощью цепного правила:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w_1}$$

$\frac{\partial L}{\partial y}$ — это просто производная функции потерь, вычисляется достаточно

легко. Найдем производную выхода слоя по параметру $\frac{\partial y}{\partial w_1}$.

Пусть

$$z_1 = f_2 \circ \dots \circ f_n(x)$$

Тогда:

$$y = f_1 \circ f_2 \circ \dots \circ f_n(x) = f_1(z_1) = z_1 W_1 + b_1$$

Поскольку y — это вектор, то только одна его компонента зависит от элемента $w_1 = (W_1)_{ij}$, и это компонента y_j . Вычислим её:

$$y_j = \sum_k z_{1_k} (W_1)_{kj} + b_{1_j}$$

Теперь найти $\frac{\partial y}{\partial w_1}$ достаточно просто:

$$\frac{\partial y_k}{\partial w_1} = 0 \quad \forall k \neq j,$$

$$\frac{\partial y_j}{\partial w_1} = z_{1_i}$$

Значит, векторная производная:

$$\frac{\partial y}{\partial w_1} = z_{1_i} e_j,$$

где e_j — это единичный вектор, все компоненты которого, кроме компоненты на j -ой позиции, равны 0.

Теперь аналогичным образом попробуем найти $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_2}$$

Разберем это выражение по порядку. Производная $\frac{\partial z_1}{\partial w_2}$ нам знакома — это производная выхода второго слоя по параметру, мы только что вычисляли аналогичную производную для первого слоя, дальнейшие математические выкладки остаются внимательному читателю в качестве самостоятельной работы.

А производная $\frac{\partial L}{\partial z_1}$ нам не знакома. Снова воспользуемся цепным правилом:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z_1}$$

Производную $\frac{\partial L}{\partial y}$ мы уже считали. А вот $\frac{\partial y}{\partial z_1}$ — это производная выхода слоя f_1 по его входу (а вход f_1 слоя — это выход f_2 слоя). Таким образом становится понятно, что:

1. На каждом слое, исключая f_n , нам необходимо считать 2 производные $\frac{\partial L}{\partial w}$ и $\frac{\partial L}{\partial z}$ — производную по параметру, которая нам нужна для вычисления градиента, и производную по входу, чтобы с помощью нее вычислить производные следующего слоя.
2. Невозможно вычислить производные слоя f_k , не вычислив производные слоев $f_1, f_2, \dots, f_{k-1}$. Это ограничение задает строгое направление обратного распространения ошибки.
3. Для вычисления $\frac{\partial L}{\partial w_k}$, производной на k -ом слое, нам необходимо найти произведение $\frac{\partial L}{\partial y} \frac{\partial y}{\partial z_1} \frac{\partial z_1}{\partial z_2} \frac{\partial z_2}{\partial z_3} \dots \frac{\partial z_{k-2}}{\partial z_{k-1}}$.

Если каждая производная $\frac{\partial y}{\partial z}$ этого произведения будет меньше 1 по модулю, например $\frac{\partial y}{\partial z} = 0.5$, то для седьмого слоя мы получим множитель $(0.5)^6 \frac{\partial L}{\partial y} \approx 0.016 \frac{\partial L}{\partial y}$.

Эта проблема называется проблемой угасающих градиентов (или исчезающих градиентов). В некоторых случаях возможна противоположная ситуация — градиенты "взрываются". Из-за этого мы (пока) не можем обучать по настоящему глубокие сети.

Для вычисления на компьютере алгоритма обратного распространения ошибки строится граф вычислений. Узлы этого графа — простейшие операции, такие как сложение, умножение, деление и тд, а ребра отображают их последовательность. Мы можем создавать собственные операции, объединяя однотипные элементы в один узел, тем самым упрощая граф. Во время прямого распространения выполнение операций осуществляется последовательно, а при вычислении градиента — в обратном порядке.

1.3.3 Градиентный спуск

Вычислив градиент ∇L_w , мы можем обновить параметры нейронной сети с помощью градиентного спуска.

Стандартный алгоритм градиентного спуска вычисляется по всей выборке:

$$w_{t+1} = w_t - \lambda \nabla L_{w_t},$$

$$\nabla L_{w_t} = \sum_{i=1}^N \nabla l_{w_t}(y_i, \hat{y}_i)$$

где N - количество объектов выборки.

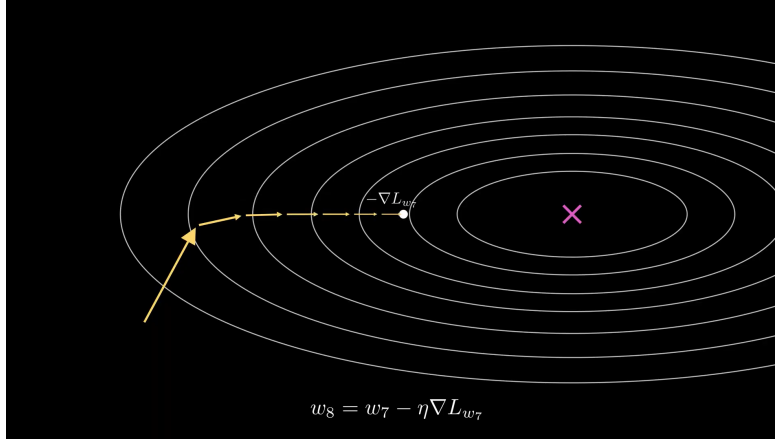


Рис. 8: Траектория градиентного спуска.

Очевидный недостаток такого подхода — потребность в огромном количестве памяти и вычислительных ресурсов. Не самый очевидный недостаток — высока вероятность попадания в локальные минимумы или седловые точки функции ошибки.

Стохастический градиентный спуск

Решение — реализуем градиентный спуск не по всему набору данных, а только по его части — по мини-батчу. Объекты в мини-батче выбираются случайным образом, делая процесс стохастическим. Такой оптимизатор называется стохастическим градиентным спуском (SGD).

$$w_{t+1} = w_t - \lambda \nabla L_{w_t},$$

$$\nabla L_{w_t} = \sum_{i=1}^B \nabla l_{w_t}(y_i, \hat{y}_i)$$

где B - количество объектов минибатча.

Реальные данные чаще всего шумные, а значит и градиент при вычислении SGD также будет шумным. Чем больше размер батча, тем точнее градиент. Но также этот шум позволяет избегать седловых точек и некоторых локальных минимумов.

Momentum

Градиентный спуск можно представить как шарик, скатывающийся по поверхности функции. Шарик, будучи объектом физическим, обладает скоростью. Реализация "скорости" в SGD (через добавление изменения весов за предыдущий шаг) называется momentum:

$$w_{t+1} = w_t - \lambda \nabla L_{w_t} + \alpha(w_t - w_{t-1})$$

Это выражение можно записать иначе:

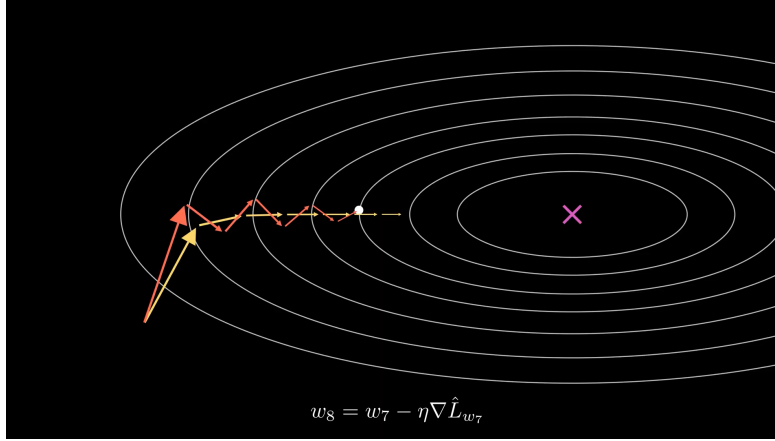


Рис. 9: Траектория стохастического градиентного спуска.

$$\begin{cases} v_{t+1} = \alpha v_t - \lambda \nabla L_{w_t}, \\ w_{t+1} = w_t + v_{t+1} \end{cases}$$

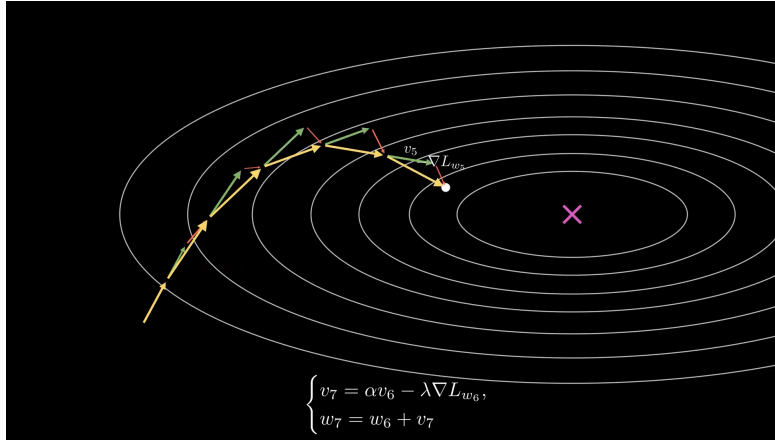


Рис. 10: Траектория momentum.

Nesterov momentum Для momentum существует модификация — Nesterov momentum. Его идея заключается в том, чтобы считать градиент не в текущей точке, а в точке, в которую попали бы, следуя импульсу:

$$\begin{cases} v_{t+1} = \alpha v_t - \lambda \nabla L_{w_t + \alpha v_t}, \\ w_{t+1} = w_t + v_{t+1} \end{cases}$$

Данный момент как бы “заглядывает в будущее”, корректируя ошибку оптимизации на текущем шаге.

Скорость обучения

Вычисление градиента — дорогая задача, поэтому хотелось бы найти оптимальные параметры нейронной сети за как можно меньшее число шагов градиентного спуска. Конечно, momentum и nesterov momentum ускоряют обучение сети, однако на скорость обучения больше всего влияет гиперпараметр λ . Чем больше λ , тем быстрее обучается модель. Однако в конце обучения хотелось бы, чтобы шаг был как можно меньше, чтобы добиться большей точности.

Пусть λ — это некоторая функция $\lambda(t)$. Тогда возможно просто подобрать подходящую функцию, чтобы значение λ соответствовало нашим желаниям. Такая техника называется **learning rate schedule**.

Стоит отметить, что параметры модели инициализируются случайным образом и градиенты на первых шагах могут быть слишком шумными. Тогда, большой шаг обучения может отбросить параметры в “плохую” часть функции потерь. Поэтому, часто скорость обучения запускают с малых, околонулевых, значений, постепенно, разогревая до “пика”, после чего она начинает затухать. Такая техника называется **warm-up**.

Adagrad

Learning rate schedule не идеален, поскольку основан на предположении, что заранее известно, на каких участках шаг обучения должен быть большим, а на каких — малым. Поэтому существуют методы адаптивной настройки скорости обучения. Первый такой метод — **Adagrad**:

$$\begin{cases} r_{t+1} = r_t + \left(\nabla L_{w_t} \right)^2, \\ w_{t+1} = w_t - \frac{\lambda}{\sqrt{r_{t+1} + \epsilon}} \nabla L_{w_t} \end{cases}$$

Этот метод старается компенсировать слишком большие и слишком малые компоненты градиента.

RMSProp

Модификация Adagrad — RMSProp (root mean square propagation). Идея этого оптимизатора заключается в том, чтобы не складывать градиенты, а брать скользящее среднее:

$$\begin{cases} r_{t+1} = \beta r_t + (1 - \beta) \left(\nabla L_{w_t} \right)^2, \\ w_{t+1} = w_t - \frac{\lambda}{\sqrt{r_{t+1} + \epsilon}} \nabla L_{w_t} \end{cases}$$

Adam

Дело осталось за малым — объединить RMSProp и momentum и получить Adam (Adaptive momentum):

$$\begin{cases} v_{t+1} = \beta_1 v_t + (1 - \beta_1) \nabla L_{w_t}, \\ r_{t+1} = \beta_2 r_t + (1 - \beta_2) \left(\nabla L_{w_t} \right)^2 \end{cases}$$

Поскольку v_0 и r_0 инициализируются нулями, то в их оценке есть смещение. Для их коррекции в Adam присутствует следующий шаг:

$$\begin{cases} \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_1}, \\ \hat{r}_{t+1} = \frac{r_{t+1}}{1 - \beta_2} \end{cases}$$

Следом идёт само обновление весов:

$$w_{t+1} = w_t + \lambda \frac{\hat{v}_{t+1}}{\sqrt{\hat{r}_{t+1} + \epsilon}}$$

Adam является серебряной пулей при оптимизации параметров нейронной сети.

1.3.4 Регуляризация

Нейронные сети обучаются на некотором тренировочном наборе данных, однако работать им приходится с теми данными, которые они, скорее всего, не видели (тестовые данные). При вычислении функции потерь на разных наборах данных возможны следующие сценарии: высокая ошибка как на тренировочном, так и на тестовом наборах данных, либо высокая ошибка на тестовых данных, но на тренировочных — почти нулевая. Говорят, что в первом случае модель недообучилась, а во втором — переобучилась.

Пусть данные генерируются с помощью следующего генератора:

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Поскольку реальные данные шумные, то всегда будет присутствовать шум, который невозможно предсказать на тестовых данных. Высокая тестовая ошибка может быть результатом недостаточной “подгонки” модели, если она слишком простая (или мало обучалась) и не способна улавливать связи в данных. Также, тестовая ошибка может быть обеспечена слишком точным следованием за шумом на тренировочных данных, из-за чего итоговая функция не способна дать правильное предсказание на тестовых данных.

Среднеквадратичная ошибка может быть разложена на три составляющие:

$$\text{expected loss} = (\text{bias})^2 + \text{variance} + \text{noise}$$

Bias (смещение) — это систематическая ошибка модели, а **variance** (разброс) — оценка чувствительности модели к изменчивости данных. Недообучение выражается в высоком смещении и низком разбросе. Переобучение — в высоком разбросе и низком смещении. Хорошо обученная модель — компромисс между смещением и разбросом (**bias-variance tradeoff**).

Чтобы уменьшить bias, нужно увеличить количество параметров модели.

Уменьшить variance возможно через уменьшение сложности модели. В этом помогают методы регуляризации. При этом будет увеличиваться bias. Регуляризация позволяет достичь компромисса для более сложной модели на тех же данных. При обучении нейронных сетей чаще всего используются l_1 и l_2 регуляризации (в общем случае - **weight decay**), идея которых заключается в добавлении нормы весов как дополнительного компонента функции потерь:

$$L_{reg_1} = L + c\|w\|_1,$$

$$L_{reg_2} = L + c\|w\|_2^2,$$

Weight decay “сдерживает” веса нейронной сети у нуля, уменьшая тем самым сложность модели. Сложность модели уменьшает и другой метод регуляризации, называемый **dropout**. Основная идея dropout — зануление некоторых выходов слоя случайным образом.

Чтобы понизить variance, не уменьшая сложность модели, необходимо расширить набор данных. Если добавить новые объекты не представляется возможным, можно воспользоваться **аугментацией данных**.

1.3.5 Нормализация

Данные имеют разный масштаб. Некоторые признаки могут быть близки к нулю, некоторые — колебаться вблизи некоторого числа. Большие по модулю значения модель будет считать важнее, даже если это не так. Для избежания этого данные нормализуются:

$$\tilde{x}^i = \frac{x^i - \mu^i}{\sigma^i + \epsilon}, \quad \mu^i = \frac{1}{N} \sum_{n=1}^N x_n^i, \quad \sigma^i = \sqrt{\frac{1}{N} \sum_{n=1}^N (x_n^i - \mu^i)^2}$$

Однако, распределение входов каждого слоя модели (кроме первого) также будет изменяться на каждом шаге обучения, поскольку изменяются параметры модели. Это означает, что каждому слою приходится заново переучиваться на новое распределение. Для решения этой проблемы применяется нормализация по батчу:

$$\hat{z} = \frac{z - \mu_B}{\sigma_B + \epsilon}$$

Но нормализуя данные таким образом, мы уменьшаем предсказательную способность сети. Поэтому результат ремасштабируется с помощью обучаемых параметров γ и β :

$$\tilde{z} = \gamma \hat{z} + \beta$$

Позже, интерпретацию с изменением распределений входов слоев стали считать недостаточной. Было выяснено, что эффект успех нормализации заключается в том, что поверхность функции потерь становится более гладкой, а градиенты - более предсказуемыми.

Часть I

Трансформеры

2 Токенизация

Нейронные сети не способны обрабатывать текстовые данные в “сыром” виде, поскольку спроектированы для работы с числовыми тензорами. Текст представлен последовательностью символов, таких как буквы, цифры, пробелы, знаки пунктуации и специальные символы. Какие-то из этих элементов встречаются чаще, какие-то — реже. Сами тексты, очевидно, могут быть разной длины.

Процесс разбиения текста на последовательность элементов называется **токенизацией**, а сами эти элементы — токенами. Все уникальные токены сводятся в словарь, где им присваивается собственный идентификатор. Это обеспечивает важное свойство токенизации — обратимость. Благодаря этому свойству, исходный текст в неизменном виде может быть декодирован из последовательности токенов.

2.1 Посимвольная токенизация

Самой первой идеей, которая приходит в голову, как разделить текст на последовательность, является посимвольное разбиение. Исходный текст итак можно представить в виде последовательности идентификаторов. Например, мы можем взять таблицу кодировки utf как словарь.

Поскольку уникальных символов немного (за исключением некоторых языков), размер итогового словаря будет небольшой, что благоприятно отразится на размере модели. Однако, если такой подход и будет успешен при обработке иероглифических языков, таких как китайский, японский или, например, древнешумерский, то при работе с другими языками могут возникнуть проблемы. При разбивке текста на символы, количество токенов может быть в десяток раз больше, чем количество слов. Также, стоит учитывать, что в отличие от китайского иероглифа, буква латинского или кириллического алфавита почти не несёт семантической (то есть смысловой) ценности.

2.2 Токенизация по словам

Посимвольная токенизация приводит к очень большой длине входной последовательности. Как решить эту проблему? Попробуем разбивать текст не по символам, а по словам. Хотя это решение и значительно сократит размер последовательности, а каждый токен будет иметь большую семантическую ценность, из-за большого количества различных форм слов, а также множества редких слов, размер словаря может раздуться до миллионов значений.

Конечно, редкие слова можно заменять одним специальным токеном, предназначенным для неизвестных элементов. Однако, это мало того, что всё еще не решает проблему огромного словаря, так и при обработке текста может потеряться смысл этих редких слов.

2.3 Byte-pair encoding (BPE)

Очевидно, что для эффективной обработки текста необходим компромиссный вариант между посимвольной токенизацией и токенизацией по словам, сочетающий преимущества обоих вариантов и минимизирующий их недостатки. Одним из таких компромиссных вариантов стал **ВРЕ-токенизатор**.

Мы хотим, чтобы часто повторяющиеся части слов были одним токеном. Возьмём эту идею за основу. Сначала разобьём каждое слово на символы и добавим сепаратор конца слова. Словарь в начале состоит из символов стартового алфавита, сепаратора конца слова и специальных токенов: <PAD>, <BOS>, <EOS>, <UNK>.

Далее, на большом корпусе текста необходимо найти самую частую пару соседних токенов. Например, для текста

Byte-pair encoding tokenization

если разбить слова на символы и добавить сепаратор '</w>', то получим пары соседних токенов:

'By', 'yt', 'te', 'e-', 'p', 'pa', 'ai', 'ir', 'r</w>';
'en', 'nc', 'co', 'od', 'di', 'in', 'ng', 'g</w>';
'to', 'ok', 'ke', 'en', 'ni', 'iz', 'za', 'at', 'ti', 'io', 'on', 'n<w>'.

Пара 'en' встретится 2 раза, а остальные — по 1 разу. Поэтому добавим токен 'en' в словарь и запомним merge-правило, после чего повторим эту последовательность шагов до того, как размер словаря не достигнет желаемого. При запоминании merge-правил крайне важно учитывать их последовательность.

Предположим, что мы обучили ВРЕ-токенизатор, но как с его помощью токенизировать текст? Добавленные элементы начального словаря определённо будут подсловами токенов в словаре. Да и неизвестно, какого размера будет каждый токен, чтобы найти его. Для этого мы и запоминали merge-правила. Чтобы токенизировать текст, применим merge-правила по порядку, в котором их сохраняли.

Алгоритм обучения и токенизации может показаться очень долгим. Однако, токенизацию текста можно применять параллельно по участкам текста, например по предложениям. Обучение ВРЕ также можно частично распараллелить: на разных участках корпуса параллельно подсчитывать частоты пар, а затем складывать их. Само обучение остается итеративным, поскольку после каждого merge-правила частоты пар меняются.

ВРЕ-токенизация предлагает компромисс между посимвольной токенизацией и токенизацией по словам, разбивая текст на последовательность подслов. В свою очередь оказывается, что такие подслова часто являются

морфемами: приставками, суффиксами, корнями, окончаниями слов. Также, ВРЕ-токенизатор способен запоминать имена и названия как отдельные токены.

Byte-level ВРЕ токенизация

Пусть мы хотим обучить ВРЕ-токенизатор для русского языка. Для этого взяли корпус сочинений Льва Николаевича Толстого (предварительно удалив отрывки на французском языке). Однако, когда модель с обученным токенизатором будет использована в чат-боте, то она может столкнуться с неожиданным входом: множество эмодзи и особые символы. А что делать, если мы хотим обучить мультязычный токенизатор?

Byte-level ВРЕ токенизатор решает проблему тем, что сначала переводит текст в байты и работает уже не с набором символов, а с набором байтов. Пробел кодируется одним байтом. Такой токенизатор подойдет как для кириллицы, так и для латиницы, эмодзи и даже бинарных файлов.

2.4 Embedding-слой

После токенизации, заменим токены в тексте на их индексы. Технически, уже эту последовательность можно обрабатывать нейронной сетью. Однако, идентификаторы токенов обладают “псевдопорядком”, из-за этого, например, токен ‘tok’ с номером 29 будет ближе к токenu ‘byte</w>’ с номером 28, чем к токenu ‘en</w>’ с номером 239. Хотя вряд ли можно говорить о какой то их особой смысловой близости, или, тем более, о том, что один токен почему-то больше другого.

При работе с табличными данными для обработки качественных признаков часто используется one-hot-encoding. Но в текущей ситуации он вряд ли применим, поскольку потребует создания гигантской квадратной матрицы с количеством строк и столбцов равными длине словаря. К тому же, эта матрица будет крайне разреженной, то есть состоять в основном из нулей, с редкими единицами.

Вместо разреженной матрицы попробуем обучить плотную матрицу — **embedding-слой**. В этой матрице вектор соответствует тому токenu, чей индекс равен индексу этого вектора. Эти вектора (а вернее весь embedding-слой) инициализируются случайным образом и обучаются вместе с остальными параметрами модели.

Приятным бонусом embedding-слоя является то, что токены, часто встречающиеся в похожих контекстах, например ‘король’ и ‘монарх’, нередко имеют близкие вектора — то есть угол между ними будет мал. Из-за этого возможна арифметика векторов. Например, если из токена ‘король’ вычесть токен ‘мужчина’ и добавить токен ‘женщина’, то получится токен, очень близкий к токenu ‘королева’. Другими словами, embedding-слой отображает токены в **семантическое пространство**.

3 Трансфомер

3.1 Преобразование одной последовательности в другую

Рассмотрим задачу преобразования одной последовательности в другую (seq2seq). Это может быть, например, машинный перевод или чат-бот, который должен выдавать ответ на вопрос. Какие модели обработки последовательностей нам пока известны?

Это RNN, LSTM и GRU, а также одномерные свёрточные нейронные сети. Стоит также вспомнить архитектуру кодировщик-декодировщик (encoder-decoder). В ней один блок, кодировщик, кодирует исходную последовательность в контекстное представление, затем другой блок, декодировщик, опираясь на него генерирует выходную последовательность token за tokenом, пока не будет выведен token $\langle \text{EOS} \rangle$.

Такая архитектура позволяет естественно работать с входными и выходными последовательностями разной длины.

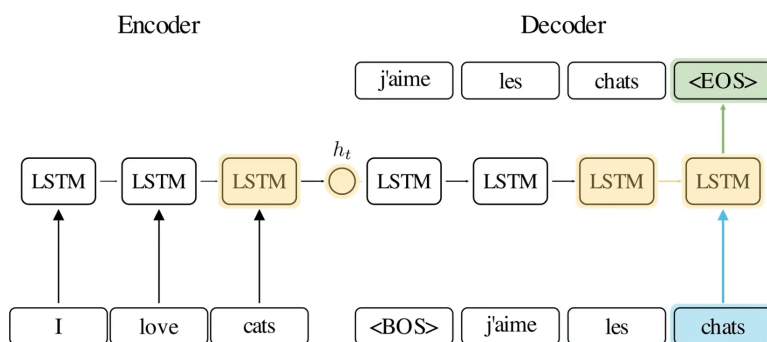


Рис. 11: Архитектура LSTM кодировщик-декодировщик.

Проблема такого подхода в том, что при обработке токенов тяжело учитывать дальние зависимости. Для решения этой проблемы в рекуррентные архитектуры стали добавлять механизмы внимания (attention), позволяющие модели обращаться к различным частям входной последовательности напрямую.

Эти методы стали в итоге state-of-the-art в задачах обработки последовательных данных. Однако они имеют существенные недостатки: подобные архитектуры сложно масштабировать, а их вычисления трудно распараллелить. Из-за своей рекуррентной природы такие сети вынуждены обрабатывать токены последовательно, поскольку за счёт этой архитектурной особенности закладывается индуцированная предвзятость (inductive bias) о порядке токенов в обрабатываемом тексте.

А что если мы откажемся от рекуррентности вообще и сделаем ставку только на механизм внимания? Вдруг окажется так, что внимание — это всё, что нам нужно? В 2017 году была предложена архитектура, которая полностью отказывается от рекуррентности и свёрток и полагается только на механизмы внимания. Эта архитектура называется трансформер.

3.2 Внимание (Attention)

Рассмотрим основу архитектуры трансформеров — внимание.

3.2.1 Мультипликативное внимание (Multiplicative attention)

Что вообще такое внимание? Этим термином будем обозначать силу связи между элементами (векторами) двух последовательностей — $G = \{g_i\}_{i=1}^{s_1} \in \mathbb{R}^{s_1 \times D}$ и $H = \{h_j\}_{j=1}^{s_2} \in \mathbb{R}^{s_2 \times D}$. Сделаем упрощение, что вектора этих последовательностей находятся в общем семантическом пространстве. Тогда для векторов, g_i и h_j , соответствующих хорошо согласованным или близким по смыслу элементам, скалярное произведение $g_i h_j^\top$ будет большим.

На основе скалярного произведения строится механизм мультипликативного внимания:

$$e_{ij} = g_i h_j^\top$$

Чтобы придать большую выразительную силу этому механизму, мы можем использовать билинейную форму:

$$e_{ij} = g_i W h_j^\top,$$

где:

$$W \in \mathbb{R}^{D \times D},$$

или же в матричном виде:

$$E = G W H^\top$$

Так мы можем учитывать, какие компоненты в семантическом пространстве имеют больший вес на матрицу оценок внимания E , а какие — меньший.

Естественно, мы не будем вручную проставлять веса для различных компонент, а сделаем матрицу W обучаемой матрицей весов.

3.2.2 Самовнимание (Scaled-dot Self-Attention)

Вывод самовнимания Связь между векторами g_i и h_j не всегда имеет смысл моделировать одной матрицей W . Выполним небольшой трюк: наложим на W условие возможного разложения на две матрицы такой же размерности, назовём их W_Q и W_K . Тогда:

$$W = W_Q W_K^\top$$

где:

$$W_Q \in \mathbb{R}^{D \times D}, \quad W_K \in \mathbb{R}^{D \times D},$$

Перепишем выражение мультипликативного внимания следующим образом:

$$E = G W H^\top = G (W_Q W_K^\top) H^\top = (G W_Q) (W_K^\top H^\top),$$

поскольку для матриц верно следующее свойство

$$B^\top A^\top = (AB)^\top,$$

то

$$E = (G W_Q) (H W_K)^\top$$

До сих пор мы рассматривали внимание между двумя различными последовательностями векторов G и H . Поскольку наша цель — решить задачу обработки одной последовательности $X = \{x_i\}_{i=1}^s$, то будем вычислять не внимание между двумя разными последовательностями, а самовнимание (self-attention) внутри последовательности X :

$$E = (X W_Q) (X W_K)^\top$$

Пусть $X W_Q = Q$ и $X W_K = K$. Тогда получим матрицу оценок самовнимания последовательности X

$$E = Q K^\top$$

Матрица весов внимания Каждая строчка матрицы E отвечает за один вектор q_i , который смотрит на все векторы $\{k_j\}_{j=1}^s$, и дает оценку, насколько сильно вектор q_i должен обратить внимание на вектор k_j

$$e_{ij} = q_i k_j^\top$$

для фиксированного q_i мы получаем строку оценок

$$e_i = (e_{i1}, e_{i2}, \dots, e_{is})$$

применим к этой строке функцию Softmax, чтобы получить нормированное распределение весов внимания:

$$a_i = \text{Softmax}(e_i)$$

или же в матричной форме:

$$A = \text{Softmax}(QK^\top),$$

где Softmax применяется независимо по строкам.

В такой форме мы избегаем появления отрицательных чисел, все значения нормализованы к интервалу $(0, 1)$, то есть находятся в одном масштабе, и $\sum_j a_{ij} = 1$.

Матрицу весов внимания можно трактовать следующим образом: у каждого вектора q_i есть 'бюджет' внимания, равный 1. Он может построить распределение этого 'бюджета' между всеми векторами $\{k_j\}_{j=1}^s$.

Нормализация матрицы внимания Для двумерного случая, функция $\text{Softmax}(x)$ почти то же самое, что и сигмоида — $\sigma(x)$. Как мы помним, основная проблема $\sigma(x)$ заключается в быстром насыщении. Softmax страдает от той же проблемы.

При вычислении матрицы внимания, нам необходимо найти скалярное произведение

$$q_i k_j^\top = \sum_{k=1}^D q_i^{(k)} k_j^{(k)},$$

отсюда видно, что при большем значении D стоит ожидать больших по модулю значений $q_i k_j^\top$. Это приведет к перенасыщению Softmax при работе с векторами большой размерности. В свою очередь, перенасыщение Softmax крайне негативно скажется на способности модели к масштабированию, поскольку стабильность обучения будет падать при увеличении размерности векторов.

Оценим среднее и дисперсию суммы

$$\sum_{k=1}^D q_i^{(k)} k_j^{(k)}$$

Поскольку

$$\mathbb{E}[q_i^{(k)}] = 0, \quad \text{Var}[q_i^{(k)}] = 1$$

и

$$\mathbb{E}[k_j^{(k)}] = 0, \quad \text{Var}[k_j^{(k)}] = 1,$$

то, сделав предположение о независимости $q_i^{(k)}$ и $k_j^{(k)}$, получаем

$$\mathbb{E}[q_i^{(k)} k_j^{(k)}] = 0, \quad \text{Var}[q_i^{(k)} k_j^{(k)}] = 1,$$

Если дополнительно предположить, что $q_i^{(k)} k_j^{(k)}$ независимы по k , то

$$\mathbb{E}\left[\sum_{k=1}^D q_i^{(k)} k_j^{(k)}\right] = 0, \quad \text{Var}\left[\sum_{k=1}^D q_i^{(k)} k_j^{(k)}\right] = D,$$

Чтобы снизить дисперсию результата скалярного произведения до 1, разделим его на $\sqrt{D}$. Таким образом получаем нормализацию

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{D}}\right),$$

которая предотвращает чрезмерное насыщение Softmax.

Перевзвешивание токенов на основе самовнимания Однако, одних весов внимания A недостаточно. Матрица A показывает лишь, каким образом каждый вектор распределяет свое внимание между остальными векторами последовательности. Однако, сама по себе, она не выполняет никаких преобразований и не говорит, какие вектора будут преобразовываться.

Самый простой вариант — агрегировать исходные вектора последовательности X :

$$O = AX$$

В этом случае, каждый вектор o_i будет являться взвешенной суммой (а поскольку $\sum_j a_{ij} = 1$, то можно говорить о средневзвешенном) исходных векторов x_j :

$$o_i = \sum_{j=1}^s a_{ij} x_j$$

Но такой подход ограничен. Как ранее в мультипликативном внимании мы ввели обучаемую матрицу W (которую потом разделили на две обучаемые матрицы W_Q и W_K), так и сейчас введем преобразование с помощью обучаемой матрицы W_V

$$V = XW_V,$$

чтобы таким образом повысить выразительность итогового преобразования

$$O = AV$$

Можно сказать о том, что матрица W_V является фильтром, определяющим с каким весом и какую информацию извлекать из каждой компоненты вектора x_j :

$$v_j = x_j W_V = \begin{pmatrix} x_{j1} \\ x_{j2} \\ \dots \\ x_{jD} \end{pmatrix}^\top \begin{pmatrix} w_{v11} & w_{v12} & \dots & w_{v1D} \\ \dots & & & \\ w_{vD1} & w_{vD2} & \dots & w_{vDD} \end{pmatrix} = \begin{pmatrix} \sum_{k=1}^D x_{jk} w_{vk1} \\ \sum_{k=1}^D x_{jk} w_{vk2} \\ \dots \\ \sum_{k=1}^D x_{jk} w_{vkD} \end{pmatrix}^\top$$

Переставим местами x_{jk} и w_{vki} и увидим, что столбцы матрицы W_V представляют собой веса взвешенной суммы по компонентам вектора x_j :

$$v_j = \begin{pmatrix} \sum_{k=1}^D x_{jk} w_{v_{k1}} \\ \sum_{k=1}^D x_{jk} w_{v_{k2}} \\ \dots \\ \sum_{k=1}^D x_{jk} w_{v_{kD}} \end{pmatrix}^T = \begin{pmatrix} \sum_{k=1}^D w_{v_{k1}} x_{jk} \\ \sum_{k=1}^D w_{v_{k2}} x_{jk} \\ \dots \\ \sum_{k=1}^D w_{v_{kD}} x_{jk} \end{pmatrix}^T$$

Таким образом, общая форма для scaled-dot self-attention выглядит следующим образом:

$$O = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V$$

Матрицы Q , K и V называются соответственно матрицами **запросов** (**queries**), **ключей** (**keys**) и **значений** (**values**).

Что такое запросы, ключи и значения Элемент матрицы оценок внимания $e_{ij} = q_i k_j^T$ представляет собой силу связи вектора запроса q_i и вектора ключа k_j . А строка $e_i = q_i K^T$ представляет собой набор оценок того, насколько сильно вектор q_i должен обратить внимание на каждый из векторов ключей последовательности.

Отсюда вектор q_i можно трактовать как **запрос**, какую именно информацию текущий элемент последовательности хочет найти в остальных элементах. А вектор k_j можно трактовать как **ключ**, по которому другие элементы последовательности могут понять, насколько данный элемент подходит их запросу.

Если запрос q_i и ключ k_j близки, то скалярное произведение получается высоким, соответственно внимание от элемента x_i к элементу x_j повышается.

Как уже было отмечено, столбцы матрицы W_V представляют собой веса взвешенной суммы по компонентам вектора x_j . Соответственно вектор $v_j = x_j W_V$ представляет собой **значения**, которые, согласно матрице W_V , должны передаваться дальше.

3.2.3 Многоглавое внимание (Multihead Attention)

Механизм самовнимания, построенный на одной тройке матриц $\{W_Q, W_K, W_V\}$ позволяет построить только одно распределение внимания. Однако, в реальных данных зависимости между элементами могут быть совершенно разными, и их может быть трудно описать с помощью одной тройки пространств ключей, запросов и значений.

Поэтому, вместо одного механизма самовнимания, будем использовать несколько параллельных и независимых механизмов — голов внимания. Так, каждая голова будет описываться своими собственными матрицами $\{W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}\}$, а значит может искать зависимости в своих собственных пространствах.

Пусть число голов внимания равно H . Для каждой головы h

$$W_Q^{(h)} \in \mathbb{R}^{D \times d_h} \quad W_K^{(h)} \in \mathbb{R}^{D \times d_h} \quad W_V^{(h)} \in \mathbb{R}^{D \times d_h}$$

где $d_h = \frac{D}{H}$, чтобы после объединения голов сохранить исходную размерность.

Запишем результат механизма самовнимания для головы h :

$$O^{(h)} = \text{Softmax}\left(\frac{Q^{(h)} K^{(h)\top}}{\sqrt{d_h}}\right) V^{(h)}$$

Объединим головы:

$$O_{\text{concat}} = \text{Concat}\left[O^{(1)}, O^{(2)}, \dots, O^{(H)}\right]$$

Выходы разных голов находятся в разных пространствах. Пусть, после объединения голов мы и получили матрицу O_{concat} той же размерности, что и исходная матрица X , информация, полученная из разных голов, все еще разделена покомпонентно. Чтобы перемешать информацию из разных голов, добавим еще одно обучаемое линейное преобразование $W_O \in \mathbb{R}^{D \times D}$:

$$O = O_{\text{concat}} W_O$$

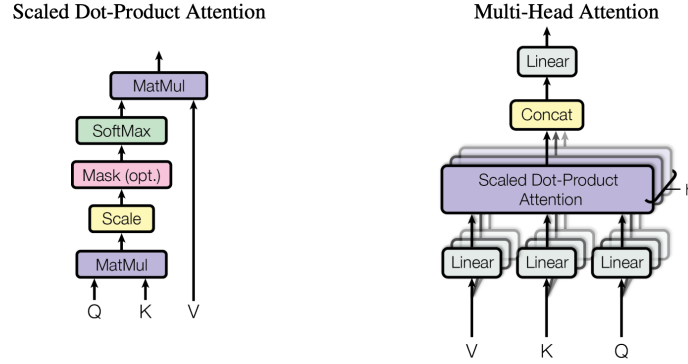


Рис. 12: Сравнение механизма самовнимания и многоглавого внимания. Взято из оригинальной статьи “Attention is all you need”.

На первый взгляд может показаться, что механизм многоглавого внимания требует значительно больше вычислений и параметров, чем механизм самовнимания с одной головой. На деле, увеличение сложности лишь связано с появлением нового слоя W_O .

3.2.4 Маскированное многоглавое внимание (Masked Multihead Attention)

До сих пор мы рассматривали лишь случай, когда каждый элемент последовательности всегда может обратить внимание на любой другой элемент. Однако, это не всегда допустимо, в некоторых задачах необходимо заранее ограничить множество токенов, на которые разрешено смотреть. Для этого в механизм внимания вводится маска.

Мы могли бы обнулять некоторые элементы a_{ij} матрицы весов внимания. Но тогда мы ломаем правило, что сумма элементов каждой строки a_i равна 1, а также сместим распределение весов внимания. Поэтому, чтобы не приходилось снова нормализовать значения весов, удобнее накладывать маску до вычисления Softmax.

Вспомним формулу Softmax:

$$\text{Softmax}_j(x) = \frac{\exp(x^{(j)})}{\sum_{k=1}^D \exp(x^{(k)})}$$

$\text{Softmax}_j(x) = 0$ тогда, когда $\exp(x^{(j)}) = 0$. А $\exp(x^{(j)}) = 0$ тогда, когда аргумент экспоненты равен $-\infty$.

Таким образом, чтобы наложить маску на матрицу весов A , необходимо к матрице оценок внимания E прибавить матрицу M , которая состоит из $-\infty$ на тех местах, которые необходимо маскировать, и из нулей во всех остальных случаях.

3.2.5 Перекрестное внимание (Cross Attention)

Ранее, при переходе от мультипликативного внимания к самовниманию, мы сделали упрощение, ограничив вход только одной последовательностью X .

$$E = (GW_Q)(W_K^T H^T) = \tilde{Q}\tilde{K}^T$$

Но, все последующие выкладки никак не противоречат тому, что последовательности $\tilde{Q}$ и $\tilde{K}$ могут происходить из разных источников и быть разной длины — $s_{\tilde{Q}}$ и $s_{\tilde{K}}$. Для корректности вычислений необходимо только, чтобы

$$s_{\tilde{V}} = s_{\tilde{K}},$$

то есть длина последовательности $\tilde{V}$ была равна длине последовательности $\tilde{K}$.

Пусть матрица $\tilde{Q}$ происходит из последовательности входов X

$$\tilde{Q} = XW_Q,$$

а матрицы $\tilde{K}$ и $\tilde{V}$ происходят из другой последовательности — Z :

$$\tilde{K} = ZW_K, \quad \tilde{V} = ZW_V,$$

Механизм внимания, построенный на основе $\{\tilde{Q}, \tilde{K}, \tilde{V}\}$ называется **перекрестным вниманием (cross-attention)**.

$$O = \text{Softmax}\left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_h}}\right)\tilde{V}$$

Перекрестное внимание позволяет вычислять внимание между разными последовательностями.

3.3 FFN-слой

Механизм внимания отвечает за обмен информацией между различными токенами входной последовательности. Но после этого еще необходимо обработать полученное представление каждого токена более сложным и нелинейным образом.

Для этого используется двуслойная нейронная сеть, называемая FFN-слоем. FFN в трансформере применяется независимо к каждому токenu входной последовательности X .

$$\text{FFN}(x_i) = \phi(x_i W_1 + b_1) W_2 + b_2,$$

где

$$W_1 \in \mathbb{R}^{D \times D_{FFN}}, \quad W_2 \in \mathbb{R}^{D_{FFN} \times D}$$

Обычно, размерность внутреннего слоя D_{FFN} выбирается больше, чем D . FFN сначала расширяет пространство признаков, а следом сжимает.

При обучении трансформера в FFN-слое используют dropout. Его обычно применяют между двумя полносвязными слоями.

$$\text{FFN}(x_i) = \text{Dropout}\left[\phi(x_i W_1 + b_1)\right] W_2 + b_2,$$

Иногда dropout применяют и после второго линейного преобразования. FFN-слой содержит большое количество параметров и выполняется независимо для каждого токена. Из-за этого возможно переобучение, которое и компенсируется регуляризацией.

3.4 Нормализация

LayerNorm и BatchNorm основаны на одной идее нормализации активаций. Разница заключается в том, как мы считаем $\mathbb{E}[X]$ и $\text{Var}[X]$. BatchNorm вычисляет эти статистики по батчу, LayerNorm же вычисляет статистики по одному объекту.

LayerNorm работает одинаково при обучении и инференсе, в отличие от BatchNorm, который требует агрегирования статистик по батчу во время обучения и использует накопленные статистики для инференса. Также,

LayerNorm, в отличие от BatchNorm, лучше применять к последовательностям разной длины и к малым батчам, так как статистики BatchNorm в этих случаях становятся более шумными.

Кроме того, LayerNorm лучше подходит для малых батчей, так как статистики BatchNorm при этом становятся слишком шумными.

3.5 Слой трансформера

Соберем все модули вместе: механизм многоглавого внимания (МНА), LayerNorm (LN) и FFN-слой. Попробуем применить их последовательно:

$$\text{TransformerLayer}(x) = \text{LN} \circ \text{FFN} \circ \text{LN} \circ \text{МНА}(x)$$

Наивный подход, когда мы применяем преобразования одно за другим, оказывается не слишком удачным для обучения. Даже один такой слой уже содержит в себе четыре последовательных матричных умножения из механизма многоглавого внимания, функцию Softmax, две нормализации, два линейных преобразования из FFN-слоя и функцию активации.

3.5.1 Остаточные связи (Residual connections)

Уже в внутри одного трансформерного слоя мы рискуем столкнуться с проблемой затухающих градиентов. А в реальной модели потребуется несколько таких слоёв. Чтобы этого избежать, к выходу каждого крупного подслоя прибавляется его вход — остаточная связь (residual connection):

$$z = \text{LN}\left(x + \text{МНА}(x)\right),$$

$$y = \text{LN}\left(z + \text{FFN}(z)\right),$$

Попробуем продифференцировать такие преобразования по входу (будем рассматривать частную производную компоненты выхода y по компоненте входа x), для второго подслоя тогда получим

$$\frac{\partial y}{\partial z} = \frac{\partial \text{LN}\left(z + \text{FFN}(z)\right)}{\partial \left(z + \text{FFN}(z)\right)} \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right),$$

а для первого

$$\frac{\partial z}{\partial x} = \frac{\partial \text{LN}\left(x + \text{МНА}(x)\right)}{\partial \left(x + \text{МНА}(x)\right)} \left(1 + \frac{\partial \text{МНА}(x)}{\partial x}\right)$$

Пусть $\tilde{z} = z + \text{FFN}(z)$ и $\tilde{x} = x + \text{МНА}(x)$. Найдем $\frac{\partial y}{\partial x}$:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial x} = \frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right) \cdot \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})} \left(1 + \frac{\partial \text{MHA}(x)}{\partial x}\right),$$

сгруппируем производные LayerNorm по входу, для этого рассмотрим частную производную:

$$\frac{\partial y}{\partial x} = \frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \cdot \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})} \cdot \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right) \cdot \left(1 + \frac{\partial \text{MHA}(x)}{\partial x}\right),$$

раскроем скобки:

$$\frac{\partial y}{\partial x} = \underbrace{\frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})}}_{\text{Градиенты почти не затухают}} \cdot \underbrace{\left(1 + \frac{\partial \text{FFN}(z)}{\partial z} + \frac{\partial \text{MHA}(x)}{\partial x}\right)}_{\text{Градиенты затухают слабее}} + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} \cdot \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты сильно затухают}}$$

Таким образом видно, что добавление остаточных связей позволяет градиентам доходить до более глубоких слоев, почти избегая затухания.

3.6 Позиционное кодирование

Рассмотрим слой трансформера. Сначала идет механизм внимания. Механизм внимания смотрит на все элементы последовательности разом, преобразование вектора никак не зависит от его порядка, а зависит от значений других векторов последовательности. Далее идет слой LayerNorm, который обрабатывает каждый токен независимо от других. Следом — FFN-слой, который также обрабатывает каждый токен независимо.

В рекуррентных сетях мы закладывали индуктивное предубеждение о порядке токенов за счёт порядка обработки элементов последовательности. В трансформере же мы обрабатываем все элементы разом. Мы нигде не добавляем информацию о порядке слов в предложении.

Между тем, порядок слов в предложении крайне важен, поскольку передает логический акцент и определяет контекст. Например, в немецком языке крайне важно располагать частицу *nicht* в нужном месте, чтобы не исказить смысл:

- Er kommt morgen **nicht** — Он не придет завтра,
- Er kommt **nicht** morgen — Он придет, но не завтра.

В этих предложениях используются абсолютно одинаковые слова, но смысл изменяется на противоположный, когда мы поменяли расположение **nicht**.

Поскольку пока мы никак не добавляем в модель информацию о позиции токенов, добавим эту информацию к самой последовательности.

3.6.1 Абсолютное позиционное кодирование

К каждому токёну прибавим некоторый вектор, который будет кодировать его абсолютную позицию в последовательности:

$$\hat{x}_{\text{pos}} = x_{\text{pos}} + \text{PE}(\text{pos}),$$

где $\text{PE} : \mathbb{N} \rightarrow \mathbb{R}^D$ — функция, отображающая позицию токёна (число) в вектор позиционного кодирования, размерность которого совпадает с размерностью вектора обрабатываемой последовательности.

Для такой функции обязательно должно выполняться то, что разные позиции должны кодироваться разными векторами. Также желательно, чтобы позиции имели близкие вектора и кодирование было таким, чтобы модель была способна восстановить относительные сдвиги между позициями.

3.6.2 Синусоидальное кодирование

Последнее особенно важно, поскольку было бы удобно, если бы кодировка позиции $\text{pos} + k$ могла быть выражена через кодировку позиции pos и величину сдвига k . Для этого естественно использовать синус и косинус, поскольку для этих функций выполняется:

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta,$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta.$$

Если теперь в некоторой компоненте вектора позиционного кодирования записать $\sin(\omega \text{pos})$, а в соседней — $\cos(\omega \text{pos})$, то для позиции $\text{pos} + k$ получим:

$$\sin(\omega(\text{pos} + k)) = \sin(\omega \text{pos}) \cos(\omega k) + \cos(\omega \text{pos}) \sin(\omega k),$$

$$\cos(\omega(\text{pos} + k)) = \cos(\omega \text{pos}) \cos(\omega k) - \sin(\omega \text{pos}) \sin(\omega k).$$

таким образом, кодировка позиции $\text{pos} + k$ в паре компонент с одной и той же частотой ω выражается через кодировку позиции pos .

Выберем набор частот, поскольку если использовать одну частоту, то разные позиции начнут периодически совпадать. Покомпонентно используются сразу многие частоты, это изменять одни компоненты быстро, а другие медленно, кодируя тем самым как локальное положение токёна, так и глобальное:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/D}}\right),$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/D}}\right),$$

где i — номер пары компонент. Малые i соответствуют низким частотам, большие i — высоким. Итоговый вектор выглядит следующим образом:

$$PE_{pos} = \begin{pmatrix} \sin\left(\frac{pos}{10000^{0/D}}\right) \\ \cos\left(\frac{pos}{10000^{0/D}}\right) \\ \sin\left(\frac{pos}{10000^{2/D}}\right) \\ \cos\left(\frac{pos}{10000^{2/D}}\right) \\ \vdots \\ \cos\left(\frac{pos}{10000^{(D-2)/D}}\right) \\ \sin\left(\frac{pos}{10000^{(D-2)/D}}\right) \end{pmatrix}^T$$

Число 10000 задает диапазон используемых частот. У каждой пары компонент свой период:

$$T_i = \frac{2\pi}{w_i} = 2\pi \cdot 10000^{2i/D},$$

так что использование различных частот для каждой компоненты также позволяет выполнить требование неравенства векторов позиционного кодирования для разных позиций.

3.7 Трансформер кодировщик и трансформер декодировщик

Рассмотрим архитектуру трансформера-кодировщика. Его задача — некоторым образом кодировать входную последовательность X_1 в последовательность Z . Архитектура представлена рядом из N_{enc} последовательных слоёв.

Каждый такой слой можно записать следующим образом:

$$\tilde{x} = \text{LN}\left(x + \text{MHA}(x)\right),$$

$$y = \text{LN}\left(\hat{x} + \text{FFN}(\hat{x})\right),$$

Теперь рассмотрим архитектуру трансформера-декодировщика. В отличие от кодировщика, декодировщик преобразует последовательность X_2 не в какое то латентное представление Z , а в другую последовательность Y , которая является продолжением последовательности X_2 . Трансформер-декодировщик предсказывает новую последовательность токенов за токеном, но об обучении трансформера немного позже.

Важно, что обычный механизм внимания не подходит для этой задачи, поскольку тогда каждый токен будет видеть все следующие, что позволяет модели переобучаться, как бы заглядывая в будущее. Поэтому будем маскировать внимание, прибавив к матрице оценок внимания маску.

В этой маске на всех позициях выше главной диагонали будет значение $-\infty$, а на главной диагонали и ниже — нули. Это позволить обнулить веса внимания для всех следующих токенов последовательности:

$$\text{MaskedMHA}(x) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

где:

$$m_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i \end{cases}$$

Запишем теперь слой трансформера-декодировщика:

$$\tilde{x} = \text{LN}\left(x + \text{MaskedMHA}(x)\right),$$

$$y = \text{LN}\left(\tilde{x} + \text{FFN}(\tilde{x})\right),$$

Также, как и кодировщик, трансформер-декодировщик можно представить рядом N_{dec} последовательных слоёв.

Теперь рассмотрим архитектуру трансформера кодировщика-декодировщика. Это более общий случай, когда необходимо преобразовать последовательность X_1 в Y . Такая задача ставится, например, при выполнении машинного перевода.

В этом случае, каждый модуль берет на себя свою задачу: кодировщик обрабатывает входную последовательность, декодировщик генерирует выходную последовательность, на основе информации из кодировщика.

Кодировщик видит входную последовательность целиком, поэтому ему достаточно обработать ее один раз. Декодировщик же будет работать авторегрессионно, последовательно генерируя токен за токеном. В этом случае одного только маскированного многоглавого внимания не хватит, поскольку декодировщику, помимо входа X_2 необходимо также учитывать и выход кодировщика Z .

Для этого в слой декодировщика добавляется перекрестное внимание:

$$\tilde{x} = \text{LN}\left(x + \text{MaskedMHA}(x)\right),$$

$$\hat{x} = \text{LN}\left(\tilde{x} + \text{CrossAttention}(\tilde{x}, Z)\right),$$

$$y = \text{LN}\left(\hat{x} + \text{FFN}(\hat{x})\right),$$

В перекрестном внимании последовательность Q происходит из входной последовательности X_2 , а последовательности K и V — из Z .

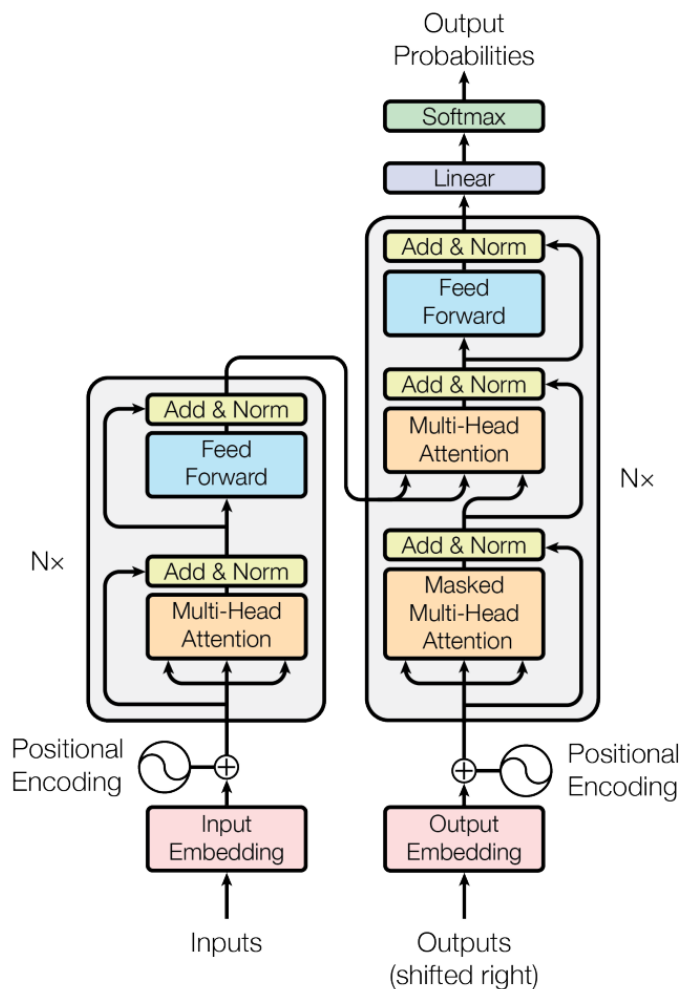


Рис. 13: Архитектура трансформера кодировщика-декодировщика, взято из оригинальной статьи “Attention is all you need”.

3.8 Обучение трансформера

Разберем обучение трансформера на примере декодировщика. Во время обучения необходимо предсказать последовательность токенов, сдвинутую на 1 шаг влево. То есть если на вход идет последовательность

<BOS>, <Архитек> <тура\w>, < >, <Трансфор>, <мер/\w>

то целевая последовательность на выходе выглядит так:

<Архитек> <тура>, < >, <Трансфор>, <мер/w>, <EOS>

Модель должна для каждой позиции входной последовательности предсказать один правильный токен из словаря. Говоря иначе, задача обучения сводится к многоклассовой классификации. Мы должны по каждому из выходных векторов предсказать следующий токен.

Для этого к выходам последнего слоя трансформера применим классификационную голову, состоящую из линейного слоя, преобразующего скрытое состояние токена в вектор логитов размерности словаря. Чтобы получить распределение вероятностей выходного токена q используем Softmax.

Далее, чтобы приблизить это распределение к целевому p , будем минимизировать кросс-энтропию $H(p, q)$ между ними.

Кросс-энтропия выступает мерой неопределенности модели в выборе следующего токена. Вспомним, что мы можем разложить кросс-энтропию на две составляющие части:

$$H(p, q) = H(p) + D_{KL}\langle p||q \rangle,$$

Пусть $\mathcal{C}$ — множество всех возможных контекстов, C — случайная величина, принимающая значения в $\mathcal{C}$, c — конкретный контекст из $\mathcal{C}$, а Y — случайная величина, соответствующая следующему токenu. Для конкретной пары (c, y) , где y — конкретный токен из Y , распределение $p(y|C = c)$ будет вырожденным. Тогда $H(p) = 0$.

Однако, если рассматривать (C, Y) как случайную пару, семплированную из истинного распределения данных, то распределение $p(y|C)$ будет отлично от вырожденного, а значит $H(p) \neq 0$.

$H(p)$ в этом случае мы можем трактовать как энтропию текста, на котором мы обучаем модель. Энтропия языка характеризует неопределенность и непредсказуемость появления следующего токена. Если бы энтропия была равна нулю, то каждый текст был бы предопределен с первого токена. В некотором смысле это шум в данных, который мы не можем устранить. И очевидно, что при обучении значение функции потерь не опустится ниже $H(p)$

$D_{KL}\langle p||q \rangle$ — это та часть, на которую мы в силах повлиять. В естественном языке существует множество правил, которые так или иначе накладывают ограничения на генерацию следующего токена. Это, например, правила грамматики, устойчивые словосочетания, синтаксические зависимости и так далее. В начале обучения модель ничего о них не знает и должна выучить их неявным образом, минимизируя $D_{KL}\langle p||q \rangle$ составляющую кросс-энтропии.

3.8.1 Perplexity

Вычисление кросс-энтропии можно представить как взвешенную сумму логарифмов вероятностей. Такое выражение тяжело интерпретировать. Переделаем его. Возьмем экспоненту от кросс-энтропии:

$$\exp(H(p, q)) = \exp\left(-\sum_i p_i \log q_i\right) = \prod_i \exp(-p_i \log q_i) = \prod_i \exp(\log q_i)^{-p_i}$$

Упростим:

$$\exp(H(p, q)) = \prod_i q_i^{-p_i}$$

Чем выше вероятность, которую модель назначает правильным токенам, тем меньше это значение. И наоборот, если модель распределяет вероятность между большим числом возможных токенов и не уверена в выборе, то значение $\exp(H(p, q))$ растёт.

Эту величину называют перплексией (perplexity):

$$\text{Perplexity}(p, q) = \exp(H(p, q)) = \prod_i q_i^{-p_i}$$

Эту величину можно трактовать как "среднее" число вариантов, между которыми колеблется модель при выборе следующего токена. Например, $\text{Perplexity} = 10$ можно интерпретировать так, будто бы на каждом шаге модель выбирает между 10 равнозначными вариантами.

Перплексию можно использовать в качестве метрики обучения языковой модели и трансформера в частности.

3.9 Инференс трансформера

Во время обучения мы подавали на вход декодировщику текстовую последовательность и учили модель предсказывать новую последовательность, сдвинутую на один шаг влево. Однако во время инференса полной текстовой последовательности ещё нет, модель должна построить её самостоятельно, токен за токеном, начиная с токена $\langle \text{BOS} \rangle$ и генерируя новые токены до тех пор, пока либо не будет достигнут токен $\langle \text{EOS} \rangle$, либо максимальная длина последовательности.

Во время обучения предсказания для всех позиций можно было считать параллельно. Во время инференса мы вынуждены все токены предсказывать последовательно, так как генерация следующего токена зависит от того, какие токены мы сгенерировали на предыдущих шагах.

На одном шаге генерации мы получаем вектор вероятностей. После получения такого вектора, необходимо выбрать токен. Самый простой вариант — использовать жадный алгоритм, всегда выбирая токен с большей вероятностью. Этот подход полностью детерминированный, что может приводить к однообразным ответам и циклам в генерации.

Другой вариант — семплировать токен из полученного распределения. Однако, в этом случае есть шанс генерации маловероятного токена, который никак не подходит под входную последовательность. Компромиссный вариант — семплировать токен из k лучших вариантов с сохранением общего распределения.

3.9.1 Температура трансформера

Для получения вероятностей слов из вектора размера `vocab_size` необходимо применить функцию `Softmax`.

При использовании `Softmax` наибольшему логиту соответствует наибольшая вероятность. Если необходимо регулировать разброс значений вероятностей, можно использовать функцию `Softmax` с температурой:

$$\text{Softmax}_T(x) = \frac{e^{x_k/T}}{\sum_{i=0}^{n-1} e^{x_i/T}}$$

Чем больше температура — тем меньший приоритет отдается большим компонентам. Это добавляет больше случайности при генерации слов в последовательности.

3.10 Масштабируемость трансформеров

Одно из главных преимуществ трансформера перед рекуррентными архитектурами заключается в том, что его вычисления хорошо параллелизируются. Это позволяет значительно ускорить обучение, используя большое количество графических процессоров (GPU).

Большая часть операций внутри трансформера — это матричные умножения (GEMM, General Matrix Multiplication). А GPU хорошо оптимизированы для вычисления матричных умножений.

Итого, параллелизм в трансформере возникает сразу на нескольких уровнях:

1. Параллелизм по батчам
2. Параллелизм по головам
3. Параллельное вычисление GEMM в механизме внимания внутри одной головы

Если раньше масштабирование количества вычислений упиралось в архитектуру модели, то теперь это чисто инженерная задача, заключающаяся в организации вычисления модели на одной видеокарте и на нескольких.

4 Vision Transformer

4.1 Проблемы свёрточных нейронных сетей

Свёрточные нейронные сети до относительно недавнего времени были стандартом в задачах обработки изображений, каждый раз являясь основой для SOTA-архитектур. Однако, как и у многих подходов, у CNN есть свои проблемы.

Во-первых, это **отсутствие восприятия глобального контекста**. На каждом слое CNN обрабатывают изображения через локальные поля. Это означает, что они рассматривают только небольшую область за раз. Такой подход ограничивает возможность сети воспринимать глобальный контекст изображения. Например, CNN не могут гарантировать того, что информация из левого верхнего угла будет связана с информацией из правого нижнего. Либо, CNN будут связывать дальние области косвенно и постепенно, что плохо скажется на обучении сети.

Во-вторых, CNN используют **одинаковые фильтры**, применяя их ко всему изображению. Такой подход может быть неэффективен, если полезная информация распределена неравномерно.

Обсудим проблему отсутствия восприятия глобального контекста свёрточными сетями. Она кажется довольно знакомой: из-за локального восприятия контекста и последовательной обработки отдельных участков сети тяжело воспринимать всю информацию целиком. Да ведь это же почти та же самая проблема, которая была решена с помощью механизма внимания! А как мы уже выяснили, наиболее удачная архитектура, основанная на внимании — это трансформер.

Если задуматься, то трансформеры решают и вторую проблему: механизм самовнимания может отдавать приоритет различным участкам при неравномерно распределённой на изображении информации, а нацеленность архитектуры трансформера на работу с последовательностями может позволить работать с изображениями различного размера.

Однако, как мы можем адаптировать архитектуру трансформера, модели изначально предназначенной для решения задачи машинного перевода, для решения задач обработки изображений? Ведь, в отличие от предложений, состоящих из сравнительно небольшого количества элементов - токенов, изображения состоят из огромного количества пикселей. Даже небольшая картинка размером 256×256 состоит из 65536 пикселей. А если изображение будет больше? Тогда не хватит никакого контекстного окна, чтобы его обработать.

Посмотрим на проблему с другой стороны: а почему мы должны считать всего один пиксель — токеном? В трансформерах после embedding-слоя токен превращается в плотный вектор (например, размерностью 512). Будет довольно странно сравнивать с ним пиксель, который для трёхканальных изображений — трёхмерный вектор. Что если попробовать подавать на вход трансформеру несколько пикселей изображения сразу, как бы нарезая его на несколько патчей? Примерно таким же вопросом в 2020м году задалась авторы модели Vision Transformer (ViT).

4.2 Токенизация изображений

Авторы ViT решали задачу классификации изображений и решили никак не менять оригинальную архитектуру кодировщика трансформера, показав, что такой подход работает лучше всего. Различия с оригинальной статьёй заключаются лишь в способе обработки данных, поступающих на вход

МОДЕЛИ.

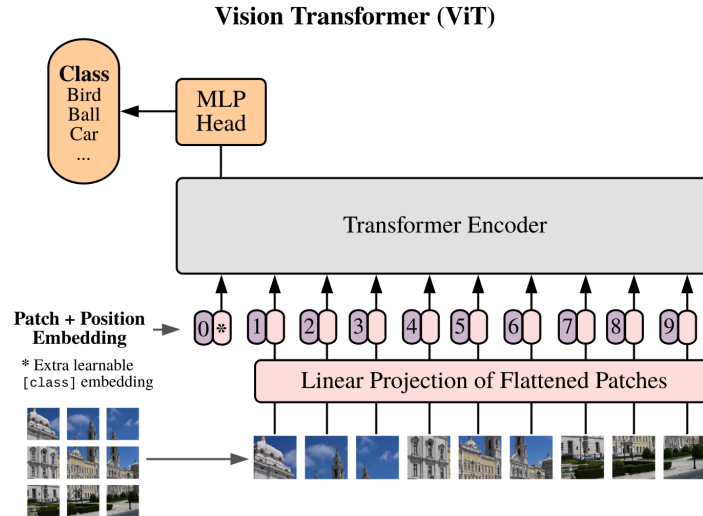


Рис. 14: Архитектура ViT. Взято из оригинальной статьи “An Image is Worth 16x16 Words”.

Для того чтобы подать изображение $x \in \mathbb{R}^{H \times W \times C}$ на кодировщик трансформера, авторы ViT предлагают разбивать его на патчи $16 \times 16 \times C$, потом проецировать в вектора размерностью D . Сделать это можно по-разному: можно выполнить с помощью линейного слоя, а можно с помощью свертки с размером ядра и шагом равными 16.

4.3 Обучаемое абсолютное позиционное кодирование

В отличие от текстовых последовательностей в задачах NLP, изображения имеют двумерную структуру. В оригинальной статье, посвященной модели трансформера, для решения проблемы позиционирования элементов последовательности использовалось одномерное позиционное кодирование. При обработке изображений логичным шагом будет применить двумерное позиционное кодирование для патчей. Авторы ViT сравнили одномерное и двумерное обучаемое позиционное кодирование и показали, что существенного прироста точности для изображений фиксированного размера от использования двумерного метода нет.

4.4 CLS токен

Если обратить внимание на схему ViT, можно заметить токен под номером 0 — токен CLS. Концепция этого токена была впервые предложена в модели BERT, используемой для задач обработки естественного языка. CLS-токен

добавляется в начало входной последовательности и служит для агрегации информации из всей последовательности, чтобы использовать её в задачах классификации. Его финальное состояние используется для агрегирования всей последовательности. Это позволяет избежать предвзятости, которая может возникнуть при выборе отдельного токена последовательности.